Optimal Correspondence of String Subsequences

Ynjiun P. Wang, *Member, IEEE,* and Theo Pavlidis, *Fellow, IEEE*

***Abstract*—The problem of substring matching when the strings are from a finite alphabet has been investigated thoroughly in the literature. The corresponding problem when the alphabet is infinite (for example strings of numbers) has received less attention. We present definitions of string distance, an effective way of computing them, and matching algorithms minimizing such distances. Our analysis also includes the matching of strings to regular expressions. We include two diverse applications: the stereo epipolar line matching and the bar code recognition.**

***Index Terms*—Matching algorithms, pattern recognition, stereo vision.**

I. Introduction

THE *partial matching* (or segment-matching [9]) problem arises in many fields of image analysis and speech recognition: for example, the correspondence problem in stereo [6], [25], the contour matching problem [28], the occluding object matching (recognition) problem [30], and syllable recognition in speech [10], etc. The general techniques for such matching task include *correlation* [5], *relaxation* [9], *time-warping* [22], and *elastic matching* [7]. However, in many one dimensional applications the cumbersome computations of relaxation and elastic matching seem unnecessary. For example, in stereo the epipolar lines correspondence problem is one-dimensional, and the contour matching problem can also be considered as a string matching (or feature vector matching) problem. It is hard to apply syntactic information to the relaxation and elastic matching techniques to do incremental matching. And the correlation technique has limitations when applied to partial matching [9]. Furthermore, we also would like the matching cost defined by the matching procedure to be a metric in pattern classification applications. The time-warping technique is not a metric [1]. These considerations motivate the development of the *optimal correspondent subsequence* problem.

The optimal correspondent subsequence (OCS) problem is derived from the classical longest common subsequence (LCS) and approximate string matching [4], [3], [15], [14], [32] problem which arises in data processing applications such as comparing two files and in genetic applications such as studying molecular evolution. The matching is defined as a notion of *exact* match or of *compatible* relation [2] on a finite alphabet. Only recently has the nonexact string matching in infinite alphabet domain received attention in the literature [12], [21]. Although in a noisy system the sampling data can always be digitized into numbers with quantization unit above the inherent noise level and represented by a finite alphabet, to handle the uncertainty of the size of finite alphabet due to the change of quantization unit conceptually is equivalent to handle an infinite alphabet. Moreover, if the size of an alphabet is too large, to maintain a huge lookup table to keep track of the interpretation of each symbol is infeasible. In this paper, the definition of optimal correspondent subsequence (OCS), which extends the finite alphabet editing error minimization matching to the infinite alphabet penalty minimization matching, is given. We also prove that the string distance derived from OCS is a metric.

An algorithm to compute the string-to-string OCS is given. The computation complexity of OCS[1] is analyzed, which is more efficient than relaxation and elastic matching for one-dimensional problems. Furthermore, an algorithm combining syntactic information in template matching is also given to show how easy it is to integrate the regular grammar into the OCS technique.

Since in different applications different penalty functions may be required, we discuss two of them: one pointwise and the other piecewise.

Finally, we discuss the application on the stereo epipolar line matching problem by using string-to-string OCS in Application A. The feasibility of applying OCS to an application, called UPC bar code [18] recognition, is investigated in Application B, and this will show the elegance of string-to-regular-expression OCS, compared to the relaxation and elastic matching techniques, on dealing with this problem.

II. Optimal Correspondent Subsequence (OCS) Problem

We define the optimal correspondent subsequence (OCS) problem as an infinite alphabet penalty minimization problem using the following entities.

Definition 1: Alphabet. The alphabet is an infinite set N which contains all characters, and lifted N, denoted as $N_\perp$, is $N \cup \{\epsilon\}$, where ϵ is a null character. By $\perp$ itself, we mean a null character or number. In many applications the reasonable choice of N is an interval of real numbers R.

Definition 2: Penalty. We say $p(x, y)$ is a penalty function, if $p : N_\perp \times N_\perp \rightarrow R$, $\forall x, y, z \in N$ such that

- $p(x, y) \geq 0$,
- $p(x, x) = 0$,
- $p(x, y) = p(y, x)$,
- $p(x, y) + p(y, z) \geq p(x, z)$.

In addition,

- $p(x, \epsilon) = p(\epsilon, x) = \tau$, where τ is a constant in R.

Notice that $p(x, y)$ is a metric over N but not over $N_\perp$.

Lemma 1: If τ is maximum (or upper bound) of R, then $p(x, y)$ is a metric over $N_\perp$.

Manuscript received May 19, 1989; revised January 23, 1990. Recommended for acceptance by R. De Mori. This work was supported by the National Science Foundation under Grant IRI-8702168 and Equipment Grant DMC-8705290, and by Symbol Technologies, Inc., Bohemia, NY.

Y. P. Wang was with the Department of Computer Science, State University of New York at Stony Brook, Stony Brook, NY 11794. He is now with Symbol Technologies, Inc., Bohemia, NY 11716.

T. Pavlidis is with the Department of Computer Science, State University of New York at Stony Brook, Stony Brook, NY 11794.

IEEE Log Number 9038390.

[1] In general, the complexity of longest common subsequence (LCS) algorithm is quadratic in time and space [32]. Hirschberg [15] gives an algorithm with linear space and quadratic time complexity. Aho, Hirschberg, and Ullman [4] have derived complexity bounds for the LCS problem and have shown that alphabet size is important. For finite alphabets an improvement on the quadratic limit should be possible. A survey of approximate string matching [14], [3] gives additional references.

0162-8828/90/1100-1080$01.00 © 1990 IEEE

Proof: We only need to verify that $p(x,y) + p(y,z) \geq p(x,z)$ for $y = \epsilon$. Since τ is the maximum, $p(x,\epsilon) + p(\epsilon,z) = 2\tau \geq p(x,z)$. Q.E.D.

Definition 3: Association function. F is an association function of string $A = a_1a_2\cdots a_n$ and $B = b_1b_2\cdots b_m$ if and only if there is a mapping $F : \{1,2,\cdots,n\} \rightarrow \{1,2,\cdots,m,\perp\}$ such that $F(i) = \perp$ or $(F(i) \neq \perp \wedge F(j) \neq \perp \wedge i < j) \Rightarrow (F(i) < F(j))$. The number of nontrivial correspondences of A and B, denoted as $\#|F|$, is the number of $F(i)$ which $F(i) \neq \perp$ and $1 \leq i \leq n$.

Now we can express the optimal correspondent subsequence problem as follows: given strings $A = a_1a_2\cdots a_n$ and $B = b_1b_2\cdots b_m$ (over alphabet N), find an association function F of A and B such that the total penalty

$$S_F(A,B) = (m+n-2q)\tau + \sum_{\forall i F(i)\neq\perp} p\big(a_i, b_{F(i)}\big) \quad (1)$$

where $q = \#|F|$, is minimized.

The LCS problem is a special case of OCS. If we set $0 \leq \tau < \epsilon$, where $\epsilon = \frac{1}{2}\min_{a_i\neq b_j}(|a_i - b_j|)$ then the F satisfying OCS will be $\forall i F(i) \neq \perp \Rightarrow a_i = b_{F(i)}$.

Roughly speaking: if τ increases, then the number of correspondences of OCS would increase. When we set $\tau \geq \sup R$ then the number of correspondences is equal to $\min(n,m)$.

Example: Given two real number strings A = 1.3 0.5 1.8 11.3, B = 1.2 1.9 11.9 9.0 5.0 and an absolute difference of two real numbers as a penalty function. If τ < 0.05, the number of correspondences is zero, i.e., $\#|F| = 0$, with minimum $S_F(A,B) = 9\tau$. If τ = 0.5, we have $F : \{1,2,3,4\} = \{1,\perp,2,3\}$ satisfying OCS, i.e., $\#|F| = 3$, with minimum $S_F(A,B) = 2.3$. If τ = 300, we have $F : \{1,2,3,4\} = \{1,2,3,4\}$ satisfying OCS, i.e., $\#|F| = 4 = \min(4,5)$, with minimum $S_F(A,B) = 313.9$.

III. String Distance

Now, let us define the distance of two strings A, B as follows:

$$D(A,B) = \min_{\forall F} S_F(A,B). \quad (2)$$

Theorem 1: The distance defined by (2) is a metric.

Proof: For strings A, B, the first two properties, $D(A,B) \geq 0$ and $D(A,A) = 0$, are obviously true.

To prove the symmetric property, we show that S_F is symmetric. From (1), the first term of the right-hand side is commutative for m and n, i.e., the order of m and n cannot effect the evaluation value, and the second term is also symmetric by the definition of the penalty function. Furthermore, the min operator is a symmetric operator, too. Therefore the symmetric property holds.

The triangular inequality can be proved by contradiction. Let us assume that F_{AB}, F_{BC}, and F_{AC} are optimal association functions corresponding to $D(A,B)$, $D(B,C)$ and $D(A,C)$, respectively, and $D(A,B) + D(B,C) < D(A,C)$. Then we can establish a new association function $F'_{AC} = F_{AB} \circ F_{BC}$ and show that $S_{F'_{AC}}(A,C) \leq D(A,B) + D(B,C) < D(A,C)$, i.e., F_{AC} is not optimal, it is a contradiction.

In detail, let $n = |A|$, $m = |B|$, and $r = |C|$; then

$$D(A,B) = (n+m-2q_{AB})\tau + \sum_{\forall i F_{AB}(i)\neq\perp} p\Big(a_i, b_{F_{AB}(i)}\Big) \quad (3)$$

$$D(B,C) = (m+r-2q_{BC})\tau + \sum_{\forall i F_{BC}(i)\neq\perp} p\big(b_i, c_{F_{BC}(i)}\big) \quad (4)$$

$$S_{F'_{AC}}(A,C) = (n+r-2q'_{AC})\tau + \sum_{\forall i F'_{AC}(i)\neq\perp} p\Big(a_i, c_{F'_{AC}(i)}\Big). \quad (5)$$

Since $\forall x,y,z \in N$, $p(x,y) + p(y,z) \geq p(x,z)$, $\Sigma_{\forall i F_{AB}(i)\neq\perp} p\big(a_i, b_{F_{AB}(i)}\big) + \Sigma_{\forall i F_{BC}(i)\neq\perp} p\big(b_i, c_{F_{BC}(i)}\big) \geq \Sigma_{\forall i F'_{AC}(i)\neq\perp} p\big(a_i, c_{F'_{AC}(i)}\big)$. And $m = q_{AB} + q_{BC} - q'_{AC} + m'$, where, $m' \geq 0$, therefore $(n+m-2q_{AB})\tau + (m+r-2q_{BC})\tau \geq (n+r-2q'_{AC})\tau$. Q.E.D.

The concept of string distance is not new. See [1] and [13] for examples. However, most of the previous work has concentrated on the finite alphabet domain. We will emphasize here the development of the penalty function in an infinite alphabet domain and elaborate the relationship between the penalty function and string distance definition.

IV. Algorithm

Suppose that we are given two strings A and B with length n and m, respectively. In order to find out the required association function F of A and B, we use a dynamic programming technique to build a minimum penalty table which accumulates the information of correspondence, and from the table we can trace back the correspondence and construct the desired F.

First, let us postulate the function $K(i,j)$ which is the minimum penalty of the match of $a_1\cdots a_i$ against $b_1\cdots b_j$. We could find the recurrence relation

$$\begin{aligned} K(i,j) = \min\{&[K(i-1,j-1) + p(a_i,b_j)], \\ &[K(i-1,j)+\tau], [K(i,j-1)+\tau]\} \end{aligned} \quad (6)$$

with $K(0,0) = 0$, and $p(a_i,b_j)$ giving the penalty of the difference between characters a_i and b_j. The three terms of this recurrence relation can be understood as: the first term a_i matches b_j, the second term leaves the a_i unmatched, and the third term allows to skip b_j without matching any character from A. The algorithm is depicted in Fig. 1. This algorithm is the extension of the Wagner and Fischer result (Algorithm X) [32]. The differences between Algorithm A and Algorithm X are: 1) the alphabet size of Algorithm A is infinite but that of Algorithm X is finite, 2) Algorithm A tries to minimize the penalty, while Algorithm X tries to construct a sequence of least editing operations to correct the Morgan editing error [26].

Computing the required $K(n,m)$ necessitates iterating over i and j, therefore it is easy to see that the complexity of Algorithm A is $O(nm)$ in time and space.

The following theorem shows that $K(n,m)$ is optimal.

Theorem 2: Given A and B strings with length n and m, respectively, and F is the association function of A and B, then

$$\forall F, K(n,m) \leq (m+n-2q)\tau + \sum_{\forall i F(i)\neq\perp} p\big(a_i, b_{F(i)}\big), \quad (7)$$

where $q = \#|F|$, i.e., $K(n,m)$ is optimal [w.r.t. all possible $F()$].

Proof: We shall prove it by double induction over i and j, which $0 \leq i \leq n$, $0 \leq j \leq m$.

Basis: $K(0,0) = 0$ is obviously optimal, and for all $i,j, K(i,0) = i\tau$, $K(0,j) = j\tau$ are optimal by the definition of OCS.

Induction Step: We assume that for all $0 \leq i \leq n-1$ and $0 \leq j \leq m-1$, $K(i,j)$ are optimal, then by definition of $K()$, we can prove by induction that $K(n-1,m)$ is optimal from the basis $K(0,j) = j\tau$. That is, it follows the basis that $K(0,m)$ is optimal and if we assume that $K(n-2,m)$ is optimal, then

```
Algorithm A(n,m,A,B,K)

1.  Initialization: K(0,0) := 0;
                    K(i,0) := i*t [i=1..n];
                    K(0,j) := j*t [j=1..m];
2.  for i:=1 to n do
    begin
3.  for j:=1 to m do
          K(i,j) := min{K(i-1,j-1)+p(A[i],B[j]),
                        K(i-1,j)+t,
                        K(i,j-1)+t}
    end
4.  return(K(n,m));
```

Fig. 1. Algorithm A (t stands for τ).

by the above assumption on $K(i,j)$ for $0 \le i \le n-1$ and $0 \le j \le m-1$ and by definition

$$K(n-1,m) = \min\{[K(n-2,m-1)+p(a_n,b_m)], [K(n-2,m)+\tau], [K(n-1,m-1)+\tau]\}$$

is optimal. Similarly we can prove that $K(n,m-1)$ is also optimal. Therefore, the $K(n,m)$, which by definition

$$K(n,m) = \min\{[K(n-1,m-1)+p(a_n,b_m)], [K(n-1,m)+\tau], [K(n,m-1)+\tau]\},$$

is optimal. Q.E.D.

Next we shall make use of the minimum penalty table, $K(i,j)$, to construct the association function F. The algorithm is similar to Algorithm Y in Wagner's paper [32] (see Fig. 2—Algorithm B).

We define the *Fset* as the set of all (i,j) such that $F(i) = j$, and for those $(i,j) \notin Fset$, $F(i) = \perp$. The proof of the correctness of this algorithm is also by induction; interested readers are referred to [32].

V. Template Matching: Integrating Regular Grammars into OCS

Usually in a recognition system a matching technique incorporates a set of templates which are used to recognize objects. Templates could be explicitly stored as a list or could be implicitly defined by rules. It turns out that it is easy to integrate a grammar into OCS and do the computation incrementally. Here we use a regular grammar to represent a set of templates (more specifically "strings"). We are going to show how to combine the regular grammar and the matching technique [11], [31] in the following paragraphs.

Suppose that we are given a regular grammar $G = (V,T,P,S)$ [16], where V is the set of variables, T is the set of terminals, P is the set of productions of the form $A \to aB$ or $A \to a$(A, B in V, a in T), and S is the start symbol. The OCS problem then becomes that of finding the sentence y in the language which is closest to the input string x. For description purposes, we will write $A \to a$ as $A \to a\#$. Using a technique similar to that introduced in the previous section, we also define a function $k(i,A)$, which is the minimum penalty of the match of $x_1 \cdots x_i$ against all strings y in the language which can be generated by starting with S and substituting using the productions until a sentential form yA is obtained. We could find the recurrence relation which is similar to (6)

$$k(i,A) = \min\Big\{\min_{\forall B:B\to aA}[k(i-1,B)+p(x_i,a)], k(i-1,A)+\tau, \min_{\forall B:B\to aA}[k(i,B)+\tau]\Big\} \quad (8)$$

with $k(0,S) = 0$, and $p(x_i,a)$ giving the penalty of the difference between the terminals x_i and a. The three terms of this recurrence relation can be understood as follows: the first term expresses the optimal matching of a single generated symbol in the language with an input symbol, the second term generates no symbols and leaves the input symbol unmatched, and the third term expresses the generation of language symbols which cannot match an input symbol. This recurrence equation is the extension of Dowling's result [11]. Although about the same time (1974) Wagner [31] also came out with a similar result, his algorithm cannot be extended to our purpose since it tightly depends on a finite alphabet lookup table.

```
Algorithm B(n,m,K,Fset)

1.  Initialization: i:= n
                    j:= m
2.  while(i != 0 && j != 0) do
    begin
3.   if K(i,j) == K(i-1,j) + t then i:= i-1
     else if K(i,j) == K(i,j-1) + t then j:= j-1
     else begin
      add (i,j) to Fset
      i:= i-1
      j:= j-1
      end
    end
```

Fig. 2. Algorithm B (t stands for τ).

Computing the optimal $k(n,\#)$ necessitates iterating over the i, starting from $k(0,S)$, and for each successive i finding $\Gamma(i)$ from the preceding $\Gamma(i-1)$, where $\Gamma(i)$ is the set of $k(i,A)$ for all variables A in V.

It is natural to implement the minimization recursive function in two stages:

1) From the first and second terms of (8), let us calculate the tentative value, $k^1(i,A)$, of $k(i,A)$.

$$k^1(i,A) = \min\Big\{\min_{\forall B:B\to aA}[k(i-1,B)+p(x_i,a)], k(i-1,A)+\tau\Big\} \quad (9)$$

2) From the third term of (8), following (9) we iterate through variables of the grammar to let values propagate until no more changes occur.

$$k^{l+1}(i,A) = \min\Big\{k^l(i,A), \min_{\forall B:B\to aA}[k^l(i,B)+\tau]\Big\} \quad (10)$$

For each i this iteration continues for at most $|V|$ cycles, leading eventually to the values $k(i,A)$ for $\Gamma(i)$.

Lemma 2: The iteration of (10) will converge within $|V|$ cycles.

Proof: Convergence Property: $\Gamma^l(i)$ is the set of $k^l(i,A)$ for all variables A in V. First we prove that if $\Gamma^{N+1}(i) = \Gamma^N(i)$ then $\forall l > N$, $\Gamma^{l+1}(i) = \Gamma^l(i)$. The definition of the iteration process, denoted as $\Psi(\Gamma^l(i))$,, is according to (10) through variables in V over the index l, i.e., $\Gamma^{l+1}(i) = \Psi(\Gamma^l(i))$. Since $\Gamma^{N+1}(i) = \Psi(\Gamma^N(i)) = \Gamma^N(i)$, we have $\Gamma^{N+2}(i) = \Psi(\Gamma^{N+1}(i)) = \Psi(\Psi(\Gamma^N(i))) = \Psi(\Gamma^N(i)) = \Gamma^N(i)$. By induction, we prove that $\forall l > N$, $\Gamma^{l+1}(i) = \Gamma^l(i) = \Gamma^N(i)$. In this case we say that $\Gamma^N(i)$ achieves a stable state.

Bounded Property: The definition of the minset of $\Gamma^l(i)$ denoted as $\overline{\min}(\Gamma^l(i))$, is the set of elements which do not change value any more through the iteration over l, i.e., $\forall l' \; \forall k^l(i,A), k^l(i,A) \in \overline{\min}(\Gamma^l(i)) \wedge l' > l \Rightarrow k^{l'}(i,A) = k^l(i,A)$.

Now we claim that $\overline{\min}(\Gamma^1(i))$ is not empty. Since the $\Gamma^1(i)$ is a finite set, there must be a lower bound, say $k^1(i,A')$, in $\Gamma^1(i)$. Furthermore, according to (10) $k^1(i,A')$ is fixed w.r.t. Ψ operation, i.e., $\forall l \geq 1, k^{l+1}(i,A') = k^l(i,A')$.

Next we claim that $\overline{\min}(\Psi(\Gamma^l(i)))$ increases at least one more element than $\overline{\min}(\Gamma^l(i))$ until $\Psi(\Gamma^l(i)) = \Gamma^l(i) \Rightarrow \overline{\min}(\Psi(\Gamma^l(i))) = \overline{\min}(\Gamma^l(i))$ which means that $\Gamma^l(i)$ achieves the stable state because of the convergence property.

To prove this, we can construct a loopless precedence graph $H(E,N)$. The elements, $k(i,B)$, in $\overline{\min}(\Gamma^1(i))$ are root nodes in N without predecessors. For each root, we create a set Q_B and put $k(i,B)$ in it. Then let all $k(i,A)$, such that $B \rightarrow aA$ and $k(i,A)$ is not in Q_B, be the descendants of $k(i,B)$ and put these elements in Q_B. Follow the above step to build the descendants of descendants of each root $k(i,B)$ until no elements can be added on.

Since there is no loop in the graph H and the longest path cannot be greater than $|V|$, it takes at most $|V|$ iterations to achieve the stable state. Q.E.D.

Example: Given a regular grammar $G = (V,T,P,S)$ where $V = \{A,B\}$, $T = \{1,2,3\}$ and $P = \{S \rightarrow 1A, A \rightarrow 2B, S \rightarrow 3B, B \rightarrow 1\#, B \rightarrow 1S\}$. An input string is 1.9 0.8 3 1. If τ is equal to 0.5, and initialize $k(0,S) = 0$, $k(0,A) = \tau$, $K(0,B) = \tau$ and $k(0,\#) = 2\tau$. Then the set $k^1(1,\{V\}) = \{k^1(1,S), k^1(1,A), k^1(1,B)\} = \{0.5, 0.9, 0.6\}$ and it is easy to verify that $k^2(1,\{V\}) = k^1(1,\{V\})$, thus $\overline{\min}(\Gamma^1(i)) = k^1(1,\{V\})$. The result $k(4,\#)$ is equal to 0.8 and the OCS is $(1,2,1,3,1) \rightarrow (\perp, 1.9, 0.8, 3, 1)$.

The following theorem gives the complexity of (8).

Theorem 3: Given an input string x with length n and $|V|$ is the number of variables of given grammar. The complexity of (8) is $O(n|V|^3)$.

Proof: To compute the second term of (10), we need to iterate at most $|V|$ times in the innermost loop. Similarly, to compute $\Gamma^l(i)$, we need to update $k^l(i,A)$ for each variable A in V which contributes to the intermediate loop of $|V|$ iterations. By Lemma 2, the outermost loop iterates over the index l at most $|V|$ times. The iteration over the index i which is the input string pointer is at most n times. Q.E.D.

Similarly, the closest sentence y can be constructed by tracing back the modified minimum penalty table entry $(k^l(i,A))$ which is defined as a record: $k'(i,A) \rightarrow c$ stores the character a generated by grammar during computing the $k(i,A)$ if the first item or the third item of (8) is the min, stores ϵ otherwise, and $k'(i,A) \rightarrow v$ stores the value of $k(i,A)$ The algorithm is shown in Fig. 3.

The boolean function *exist*(A,B,t) is defined as: it returns *TRUE* if there is a B in V such that $k'(i,A) \rightarrow v = k'(i,B) \rightarrow v + t$ and $B \rightarrow aA$ in P where a is the character stored in record $k'(i,A) \rightarrow c$; *FALSE* otherwise.

```
Algorithm D(n,k',y)

1.  Initialization: i:= n
                    A:= #
                    y:= NULL
2.  while(i != 0 && A != S) do
    begin
3.   if k'(i,A)->v == k'(i-1,A)->v + t then i:= i-1
     else if exist(A,B,t)
          then begin
           y:=k'(i,A)->c @ y
           A:=B
          end
     else begin
      y:=k'(i,A)->c @ y
      i:= i-1
      A:= B
     end
    end
```

Fig. 3. Algorithm D (t stands for τ, @ means concatenation).

VI. Penalty Functions

A. Pointwise Penalty Function

For real numbers the simplest pointwise penalty function is the Euclidean distance, $p(x,y) = |x - y|$. It is easy to check that this definition satisfies the definition of penalty (cf. Section II).

B. Piecewise Penalty Function

In some applications, the local structure matching is more important than the pointwise matching. For these cases, the piecewise penalty function is introduced.

The basic idea for the piecewise penalty function (*ppf*) is using the neighborhood information. There are many functions satisfying the definition of piecewise penalty. The following are two examples together with proofs that are well defined *ppf*.

Let us define the *ppf* $p_b(x_i, y_j)$ as follows:

$$p_b(x_i,y_j) = \sum_{k=-b}^{b-1} |(x_{i+k+1} - x_{i+k}) - (y_{j+k+1} - y_{j+k})|. \quad (11)$$

In the domain of real numbers, $p_b(x_i,y_j)$ is a metric. The first three metric properties, $p_b(x_i,y_j) \geq 0$, $p_b(x_i,x_i) = 0$, and $p_b(x_i,y_j) = p_b(y_j,x_i,)$ are easy to check. Furthermore, the above definition is a discrete version of Euclidean distance of the first derivatives of each correspondent neighbor points. Therefore, the triangular inequality holds.

Another example, if we consider these two patches around x_i and y_j as two strings. Then we can define *ppf* as an OCS with pointwise penalty function. We already proved that string distance measured by (2) is a metric. Therefore this definition meets the definition of penalty. This example gives an important view of the OCS problem, i.e., the OCS algorithm is a recursively applicable algorithm. We will find its importance in the stereo epipolar line matching later.

The definition of (11) is a two-sided piecewise penalty function. In order to take advantage of the incremental matching property of the recurrence equation (8), we would like the penalty function to be one-sided piecewised.[2] The one-sided version of above *ppf* is defined

$$p_b^-(x_i,y_j) = \sum_{k=-b}^{-1} |(x_{i+k+1} - x_{i+k}) - (y_{j+k+1} - y_{j+k})|. \quad (12)$$

It is easy to check that the above one-sided *ppf* is a metric.

[2] In the terminology of digital signal processing, this corresponds to a causal system which can filter input signal out in real time response.

VII. Revised OCS Algorithm for the One-Sided Piecewise Penalty Function

Algorithm A can be adapted to the *ppf* easily with an independent penalty function module to compute the penalty, since both string values are accessible during the run time. But the recurrence equation (8) is not obviously adapted to the one-sided *ppf*. The reason is that there is no accessible string generated by the grammar before finishing the matching. Therefore, a modification of memorizing the partial string being generated in the state variable $k(i, A)$ is needed. The revised recurrence equation (8) is as below:

$$k_\beta(i,A) = \min\Big\{\min_{\forall B:B\to aA}[k_{\gamma c}(i-1,B) + p_b^-(x_i,a)], \; k_{\gamma c}(i-1,A)+\tau, \min_{\forall B:B\to aA}[k_{\gamma c}(i,B)+\tau]\Big\} \quad (13)$$

where $\beta = a\gamma$ when the minimum occurs at the first or the third term of the right-hand side of (13), $\beta = \gamma c$ when the minimum occurs at the second term, and $|\beta| = |\gamma| + 1 = b$.

The complexity of revised OCS algorithm is increased by a constant factor, b, i.e., the order of the complexity remains the same.

VIII. Application A: The Epipolar Lines Correspondence Problem

Finding the conjugate points of two images on epipolar lines is intrinsically an OCS problem [17], [25], [6]. Several matching schemes for stereo matching have been proposed, e.g., a cooperative algorithm [24] is a neural net-like implementation. Kim and Aggarwal [20] used a relaxation method. Ohta and Kanade [27] and Lloyd [23] used dynamic programming.

Dan and Dubuisson [8] is probably the first paper applying string matching scheme to the stereo matching problem. But they used a finite alphabet, called edges-based coding, to do string matching. In our approach, we directly use the raw images (could be gray level images, or raw zero-crossing images) to do the matching without any precoding process. The use of an infinite alphabet avoids the quantization errors. In the raw images a lot of details (e.g., amplitudes around zero-crossings) are required for correct matching. These details will not be preserved by some feature coding processes, like edge-based coding. Furthermore, since the OCS algorithm is a general matching scheme, we can choose any kind of primitive, e.g., gray level, edgelet, as a matching entity.

A. Algorithm

The algorithm (see Fig. 4) for finding the conjugate points of left and right images can be described in two stages.

1) Epipolar Line Registration: For each scan line $left_i$ of the left image, search the correspondent scan line of the right image $right_{j'}$ such that the penalty of OCS $(left_i, right_{j'})$ is minimum for j in the range of (i − *maxshift*, i + *maxshift*).

2) Finding Conjugate Points: For two registered scan lines (i.e., on the same epipolar line), for each segment, $leftseg_i = x_{i-r}x_{i-r+1}\cdots x_{i+r}$ (which can be considered as a string consisting of a sequence of real-valued pixels centered by pixel x_i with radius r), of the left eye scan line, search the correspondent segment of the right eye scan line, $rightseg_j = y_{j-r}y_{j-r+1}\cdots y_{j+r}$, such that the penalty of OCS $(leftseg_i, rightseg_{j'})$ is minimum for j in the range of (i − *maxdisparity*, i + *maxdisparity*).

In Fig. 4, the Algorithm Registration (n, L, R) registers the right scan line *R*[register] to the left scan line $L[n]$ in two given *M*

```
Algorithm Registration(n,L,R)
    /*Epipolar line registration*/
    min := A(M,M,L[n],R[n-maxshift]);
    register := n-maxshift;
    for i:=n-maxshift+1 to n+maxshift do
        if min > A(M,M,L[n],R[i]) then
            min := A(M,M,L[n],R[i]);
            register := i;
    return(register);

Algorithm Correspondence(p,r,L[n],R[m])
    /*Finding conjugate points*/
    l:=2*r+1;
    min := A(l,l,&L[n][p-maxdisparity-r],
                 &R[m][p-maxdisparity-r]);
    disparity := -maxdisparity;
    for i:=p-maxdisparity+1 to p+maxdisparity do
        if min > A(l,l,&L[n][i-r],&R[m][i-r]) then
            min := A(l,l,&L[n][i-r],&R[m][i-r]);
            disparity := i-p;
    return(disparity);
```

Fig. 4. OCS stereo algorithm.

by *M* images *L*[ ] [ ] and *R* [ ] [ ]. The Algorithm Correspondence $(p, r, L[n], R[m])$ computes the disparity of the point $L[n][p]$ with respect to the scan line *R*[*m*]. The r is the matching patch radius. Both algorithms use Algorithm A to compute the matching cost.

B. Experiment

The algorithm was tested by Dr. William Sakoda on a pair of *SPOT* images at Grumman Data Systems. The goal was to construct the depth map of two 512 by 512 aerial images (see Fig. 5). The inputs were $\nabla^2 G$ filtered images with σ = 2.0. The output was a 128 by 128 depth map. The OCS algorithm replaced an earlier matching procedure which used a correlation matching algorithm.

Figs. 6 and 7 are the depth maps by using correlation method and OCS algorithm, respectively. The gray level in Figs. 6 and 7 represents the relative height, the brighter the lower and the darker the higher. Those yellow and green areas are unmatching and mismatching areas, respectively. Compared to Fig. 6's large unmatching and mismatching areas, Fig. 7's unmatching and mismatching areas have been reduced significantly.

The images (Fig. 5) can be displayed by using different color maps, and a human observer with color glasses can fuse the images to verify the depth map [19]. An interesting case is presented at the right-upper corner where there is a big unmatched area in Fig. 6. The OCS algorithm gives the correct solution in that area; see Fig. 7.

As we mentioned in Section VI-B, it is possible to apply OCS recursively (or in other sense *hierarchically*). In the aerial imagery, we can also apply *ordering constraints* [8] in the global matching (i.e., consider the epipolar lines as strings). Therefore, in phase two of Fig. 4 instead of finding a local minimum in the range of (i − *maxdisparity*, i + *maxdisparity*), we can take that as a piecewise penalty function and apply Algorithm A once more on the whole scan lines to find out the OCS and to compute the disparities accordingly. This deserves further investigation.

Fig. 5. Pair of aerial images.

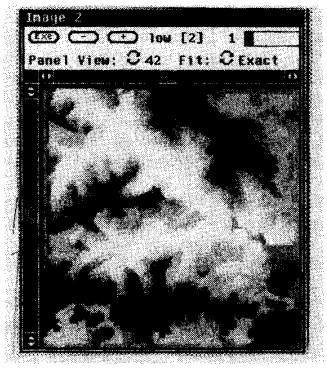


Fig. 6. Depth map by correlation method.

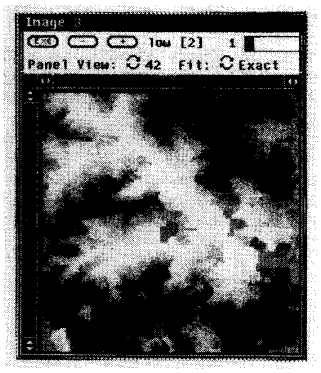


Fig. 7. Depth map by OCS algorithm.

IX. Application B: An Investigation of the Applicability of OCS to a UPC Decoding Scheme

Another application of OCS matching is to UPC bar code recognition. The information in bar codes is conveyed by the arrangement of bars and spaces (elements) of different widths. The elements are translated by a scanner from their printed form to an analog electrical signal that represents the relative widths of the elements in the symbol.

The UPC (Universal Product Code) symbol [29], [18] is usually printed on the products for identification purposes. The symbol consists of 59 elements. Every element width ranges from 1 to 4 modules. The first (last) three consecutive one module elements are leading (trailing) guard elements. There are five consecutive one module central guard elements in the center of the symbol which divide the symbol into two even blocks. Each block contains 24 elements representing 6 digits. Every four consecutive elements with fixed length of seven modules represent a digit.

We can use a regular grammar (see Fig. 8) to describe the legal UPC codes on the level of element width string input. In Fig. 8 the grammar has 288 states with sentence length of 59. The numbers $1, 2, 3, 4$ represent the module numbers. After three consecutive one module transitions, the grammar describes the first digit by 22 states. The following five rectangular boxes represent the same structure as the first digit. Then following by five consecutive central guard elements the long rectangular box represents the same grammar structure as the left-hand side 6 digit block. The symbol ends with three consecutive one module trailing guard elements.

The output of the optoelectric circuit of the bar code scanner is a sequence of numbers representing interleaving bar and space width. Since there is a noise in the scanning process, the raw data could be any occurrence of number, it is not an ideal target for decoding. In order to reduce the effects of noise, searching the best matching sentence generated by the presetting grammar is necessary. The use of an infinite alphabet in matching phase can avoid quantization errors. After the matching process, we use the matching result, i.e., the sentence closest to the input string, to decode.

Example: Data in the second column of Fig. 9 are collected from a hand-held laser scanner on scanning a UPC label forward. Because of limited space we only illustrate up to data no. 35, which is the left-side block of the UPC label including the leading guard elements, the first six digits, and the central guard elements. Let us set the module equal to 24^3 and use the grammar depicted in Fig. 8 to parse the data, and we obtain a sentence as in the third column of Fig. 9, which is the sentence closest to the data collected from the scanner. In this example, the original data is undecodable because of an error on data no. 17 into which three elements are merged. But the resulting sentence is decodable and the decoding result is in the fourth column of Fig. 9.

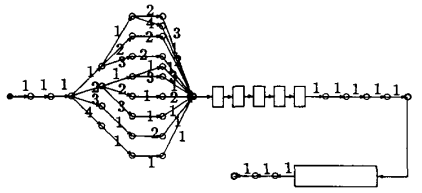
X. Conclusions

The classical LCS problem is well understood [4]. It seems that the OCS problem has not been explored as much as LCS. This paper gives the definition of the optimal correspondent subsequence (OCS) problem and the algorithms. The complexity of OCS is quadratic. It is superior to relaxation and elastic matching of which complexity are cubic. In comparison to relaxation and elastic matching techniques, the OCS also shows its elegance in combining syntactic information and doing matching incrementally. The diversity of the OCS algorithm is shown in the last two applications. It can tackle either the string-to-string or string-to-regular-grammar approximate matching problem in the infinite alphabet domain.

Acknowledgment

The authors would like to thank Dr. W. Sakoda and K. Huang for their help in using their systems and for discussions on this paper. We thank Grumman Data Systems for the testing of our algorithm in their Laboratory.

[3]This number is from finding the minimum module by histogramming method. Since it is out of the scope of this paper, we omit the discussion of this step here. Interested readers are referred to [33].



Total states = 288

Final state °
State ○
Starting state •

Fig. 8. Regular grammar used in UPC recognition.

```
MODULE = 24

No. DATA  SENTENCE DECODE          No. DATA  SENTENCE DECODE
 1   255                          19   23      1       4
 2   255                          20   26      1
 3   255                          21   71      3
 4   255                          22   51      2
 5   255                          23   70      3       0
 6   24     1    leading          24   49      2
 7   24     1    guard            25   22      1
 8   28     1    elements         26   27      1
 9   71     3       0             27   71      3       0
10   48     2                     28   47      2
11   23     1                     29   23      1
12   23     1                     30   23      1
13   25     1       4             31   25      1    central
14   26     1                     32   23      1    guard
15   71     3                     33   24      1    elements
16   46     2                     34   24      1
17   72   -- 1 a     6            35   25      1
          \  1 merge                    :
           \ 1 error                    :
18   96     4                          :
```

Fig. 9. An example of parsing UPC.

References

[1] K. Abe and N. Sugita, "Distances between strings of symbols, review and remarks," in *Proc. 6th Int. Conf. Pattern Recognition,* Oct. 1982, pp. 172–174.

[2] K. Abrahamson, "Generalized string matching," *SIAM J. Comput.,* vol. 16, no. 6, pp. 1039–1051, Dec. 1987.

[3] A. V. Aho, "Algorithms for finding pattern in strings," in *The Handbook of Theoretical Computer Science.* Amsterdam, The Netherlands: North-Holland, 1989.

[4] A. V. Aho, D. S. Hirschberg, and J. D. Ullman, "Bounds on the complexity of the longest common subsequence problem," *J. ACM,* vol. 32, no. 1, pp. 1–12, Jan. 1976.

[5] D. H. Ballard and C. M. Brown, *Computer Vision.* Englewood Cliffs, NJ: Prentice-Hall, 1982.

[6] S. T. Barnard and M. A. Fischler, "Computational stereo," *Comput. Surveys,* vol. 14, no. 4, pp. 553–572, Dec. 1982.

[7] D. J. Burr, "Elastic matching of line drawings," *IEEE. Int. Joint Conf. Pattern Recognition,* 1980, pp. 223–228.

[8] H. Z. Dan and B. Dubuisson, "String matching for stereo vision," *Pattern Recognition Lett.,* vol. 9, pp. 117–126, Feb. 1989.

[9] L. S. Davis, "Shape matching using relaxation techniques," *IEEE Trans. Pattern Anal. Machine Intell.,* vol. PAMI-1, no. 1, pp. 60–72, Jan. 1979.

[10] R. de Mori, P. Laface, and Y. Mong, "Parallel algorithms for syllable recognition in continuous speech," *IEEE Trans. Pattern Anal. Machine Intell.,* vol. PAMI-7, no. 1, pp. 56–69, Jan. 1985.

[11] G. R. Dowling and P. A. V. Hall, "Elastic template matching in speech recognition, using linguistic information," in *Proc. 2nd Int. Joint Conf. Pattern Recognition,* Aug. 1974, pp. 249–250.

[12] T. Eilam-Tzoreff and U. Vishkin, "Matching patterns in strings subject to multi-linear transformations," *Theoretical Comput. Sci.,* vol. 60, pp. 231–254, 1988.

[13] K. S. Fu, *Syntactic Pattern Recognition and Applications.* Englewood Cliffs, NJ: Prentice-Hall, 1982.

[14] P. A. V. Hall and G. R. Dowling, "Approximate string matching," *Comput. Surveys,* vol. 12, no. 4, pp. 381–402, Dec. 1980.

[15] D. S. Hirschberg, "A linear space algorithm for computing maximal common subsequences," *Commun. ACM,* vol. 18, no. 6, pp. 341–343, June 1975.

[16] J. E. Hopcroft and J. D. Ullman, *Formal Languages and Their Relation to Automata.* Reading, MA: Addison-Wesley, 1969.

[17] B. K. P. Horn, *Robot Vision.* Cambridge, MA: MIT Press, 1986.

[18] *UPC Symbol Specification,* Uniform Product Code Council, Inc., May 1982.

[19] B. Julesz, *Foundations of Cyclopean Perception.* Chicago, IL: University of Chicago Press, 1971.

[20] Y. C. Kim and J. K. Aggarwal, "Finding range from stereo images," *Comput. Vision Pattern Recognition,* pp. 289–294, 1985.

[21] D. Sankoff and J. B. Kruskal, Eds., *Time Warps, String Edits, and Macromolecules: The Theory and Practice of Sequence Comparison.* Reading, MA: Addison-Wesley, 1983.

[22] J. B. Kruskal and M. Liberman, "The symmetric time-warping problem: From continuous to discrete," in *Time Warps, String Edits, and Macromolecules: The Theory and Practice of Sequence Comparison,* D. Sankoff and J. B. Kruskal, Eds. Reading, MA: Addison-Wesley, 1983, ch. 4.

[23] S. A. Lloyd, "Stereo matching using intra- and inter-row dynamic programming," *Pattern Recognition Lett.,* vol. 4, pp. 273–277, Sept. 1986.

[24] D. Marr and T. Poggio, "Cooperative computation of stereo disparity," *Science,* vol. 194, pp. 283–287, 1976.

[25] D. Marr, *Vision.* San Francisco, CA: Freeman, 1982.

[26] H. L. Morgan, "Spelling correction in systems programs," *Commun. ACM,* vol. 13, no. 2, pp. 90–94, Feb. 1970.

[27] Y. Ohta and T. Kanade, "Stereo by intra- and inter-scanline search using dynamic programming," *IEEE Trans. Pattern Anal. Machine Intell.,* vol. 7, no. 2, pp. 139–154, 1985.

[28] T. Pavlidis, "The use of a syntactic shape analyzer for contour matching," *IEEE Trans. Pattern Anal. Machine Intell.,* vol. PAMI-1, no. 3, pp. 307–310, July 1979.

[29] D. Savir and G. J. Laurer, "The characteristics and decodability of the universal product code symbol," *IBM Syst. J.,* vol. 1, pp. 16–33, 1975.

[30] J. T. Schwartz and M. Sharir, "Identification of partially obscured objects in two and three dimensions by matching of noisy 'characteristic curves'," *Int. J. Robotics Res.,* vol. 6, no. 2, pp. 29–44, 1987.

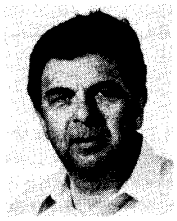
[31] R. A. Wagner, "Order-*n* correction for regular languages," *Commun. ACM,* vol. 17, no. 5, pp. 265–268, May 1974.

[32] R. A. Wagner and M. J. Fischer, "The string to string correction problem," *J. ACM,* vol. 21, no. 1, pp. 168–173, Jan. 1974.

[33] Y. P. Wang and T. Pavlidis, "Bar code recognition," Dep. Comput. Sci., SUNY at Stony Brook, Stony Brook, NY, Tech. Rep., Feb. 19, 1988.

Ynjiun P. Wang (S'88–M'89) received the B.S. degree from National Chiao Tung University, Taiwan, in 1984, and the M.S. and Ph.D. degrees from the State University of New York at Stony Brook in 1987 and 1989, respectively, all in computer science.

Since 1988, he has been an R&D Scientist at Symbol Technologies, Inc., Bohemia, NY, and is responsible for the development of a new two-dimensional bar code symbology. This involves research in optical scanning, signal processing, information theory, coding theory, and algorithms. His current research interests besides bar code symbology include spatial information theory, coding theory, image processing and analysis, and string matching algorithms.



Theo Pavlidis (S'62–M'64–SM'77–F'79) received the Diploma in mechanical and electrical engineering from the National Technical University of Athens, Greece, in 1957, and the M.S. and Ph.D. degrees in electrical engineering from the University of California at Berkeley in 1962 and 1964, respectively.

He is a leading Professor with the Department of Computer Science of SUNY at Stony Brook where he has been since 1986. He is also a scientific advisor to Symbol Technologies and a consultant to AT&T Bell Laboratories. From 1980 to 1986 he was with the Computer Science Research Center of AT&T Bell Laboratories, Murray Hill, NJ. Between 1964 and 1980 he was on the faculty of the Department of Electrical Engineering and Computer Science at Princeton University. During the 1978–1979 academic year he was a Visiting Professor of Electrical Engineering and Computer Science at the University of California at Berkeley. His industrial experience includes also consulting with RCA, various U.S. Army Laboratories, Datacopy, and other companies. His research interests are in the areas of image analysis, pattern recognition, and computer graphics. His research during 1980–1986 centered on the recognition of text and graphics together and the development of a graphics editor using image analysis techniques. More recently he has also resumed his earlier work on gray scale image analysis with emphasis on cartographics applications. In addition, he is investigating methodologies for the reading of high density bar codes. He is the author of three books, *Biological Oscillators: Their Mathematical Analysis* (New York: Academic, 1973), *Structural Pattern Recognition* (New York: Springer-Verlag, 1977), (translated into Chinese, 1981), and *Algorithms for Graphics and Image Processing* (Rockville, MD: Computer Science Press, 1982), (translated into Russian, 1986, Polish, 1987, and Chinese, 1988) and over 100 technical papers.

Dr. Pavlidis was the Editor-in-Chief of the IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE (PAMI) from 1982 to 1986, and has been a member of the editorial board of the *Proceedings of IEEE* (since 1988) and three other journals. His conference-related activities include: Chairman of the Dahlem Workshop on Biomedical Pattern Recognition and Image Processing (Berlin 1979), General Chairman of the Fifth International Conference on Pattern Recognition (December 1980, Miami Beach), Scientific Chairman of the NATO Workshop on Syntactic Pattern Recognition (October 1986, Barcelona), and General Chairman of the IEEE Robotics and Automation Conference (Philadelphia, 1988). He has been a member of various advisory councils of the National Science Foundation and he is a member of the Universities Space Research Association Science Council for Computer Science. He is a member of the Association for Computing Machinery and Sigma-Xi.